Project Proposal

Paul Robinson

October 19, 2026

Problem Description

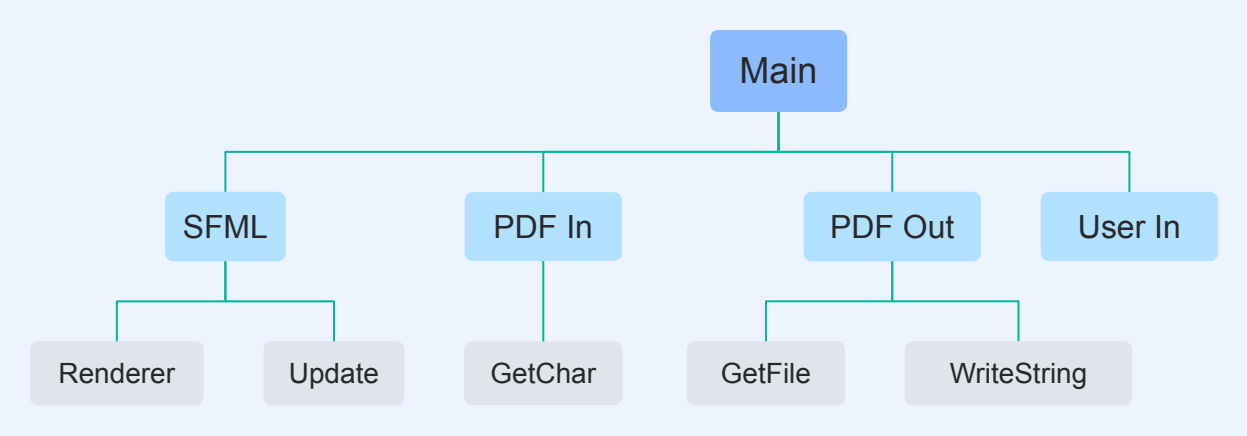
The purpose of this project is to solve a problem I have on a daily basis while playing guitar. I oftentimes find myself chaning tunings to play songs by the bands Van Halen and Stevie Ray Vaughn with Double Trouble. Their songs are oftentimes in the tuning of E flat standard, so every note is shifted up by one. This becomes a problem when I want to play a song that is written in E standard. People familiar with guitar, but unfamiliar with this problem because they don't use alternate tunings may just say, "retune your guitar or use a capo." This seems valid since both do solve the problem in a way. The problem with retuning is that my guitars have a floating point style bridge where retuning can take 10 to 30 minutes. This becomes an issue when playing in front of people and time is of the essence. Capos work partially to change the tuning, but the notes are all shifted which becomes a problem if you try to play any difficult songs by memory.

For this reason I've opted to make a problem that lets users shift notes in guitar tabulature for each string. This program allows users to pragmatically change the tuning of individual strings to easily play songs from other tunings without using a capo or to use a capo to avoid retuning guitars mid performance. This can allow someone in a tuning like D standard to learn a E standard song with notes far beyond a capo easily.

Program Data

1. PDF Data (In)
    a) This is our input data that is given by the user. Users give guitar notation to be altered while they read the music. These files can be recieved from artists, individuals, or websites based around sharing them.
2. PDF Data (Out)
    a) Users can opt to print out the data as a pdf file to save to their computer. This can make sharing possible to others or simply allow for people to bypass recompiling their files for songs they need to remember or practice after the program runs.
3. Vector Screen
    a) This class vector stores portions of the pdf to be edited rather than the whole thing at once. This allows for easier rendering as well as changes to tuning in the program for longer songs not take too long. Long songs could crash the program or just cause the program to timeout, so only updating portions of the song at a time is better. It gives O(1) for updating the song as a protion 1 rather than O(N) where N is the length of the song in screens.

Program Classes



Main

This is simply where the code starts and stiches all classes together.

Renderer (SFML)

Used to draw things on screen and get data in from user.

Renderer

Updates the screen whenever called. May switch pages of the guitar tabs or scroll.

Update()

Updates based on user input. Can modifiy the output of the guitar notation.

PDF In

Takes in data as a pdf and reads it

GetFile(String FilePath)

Gets the file from user and also makes sure it is valid, otherwise print out that it failed

GetChar(int pCursorLocation)

Gets the currently looked at character based on the location

PDF Out

Prints out if the user requests so recomilation of files is not necessary

GetFile(String FilePath)

Gets the file from user and also makes sure it is valid, otherwise print out that it failed

WriteString(int pCursorLocation)

Gets the currently looked at character based on the location and prints it out based on user wants

User In

Takes in the input from a user and acts accordingly. I honestly have no idea how this'll work with SFML since I've never used it.

PseudoCode

```
// Note if none of this makes sense to the person reading
// I'm sorry, but this is how I write pseudo code to just write
// ideas out quickly



int main() {
  //renders with SFML
  window = new.Window(Vec2(size_of_window));

  //likely switch to being given by user for both input and output
  file_in = new.FileInput("path");
  file_out= new.FileOutput("path");

  //runs until the user wants out
  while(program_running) {
    window.update();
  }

  //program ran sucessfully
  return 0;
}

//SFML Window Class
class Window(int height, int width) {
  //this screen page holds what the user is looking at
  // and allows them to look at parts of a song closeup
  // also allows for changes to certain portions of a song
  // as well as easier rendering since one screen is one display screen
  // hence the name. This is the format for the music, not the actual screen
  Vector(Screen);
}

void Window::update() {
  //update screen
  //checks if the user wants out too
  check_for_errors();
}

//gets any neccessary errors
int Window::check_for_errors() {
  // check and return errors
  return error
}

/*
 *
 * handles user input and stuff
 * likely extends another class from SFML
 *
 */
```

```
class Input();

//handles user inputs accordingly
void Input::HandleUserControl() {
  //foo
}

//handles file management of input files
//Output is the same stuff so I didn't write out the functions for output
class FileInput(String path);

//gets file and checks validity
void FileInput::GetFile();

//gets char location
char FileInput::GetChar(int location);
```

Conclusion

This project is mostly a study of how SFML works, how to format files of a type different to txt, and how to use C++ for something useful. I do not know a lot of stuff about C++ or many of the datatypes required to do this project, so learning along the way will likely be necessary.